Algorithms: Design and Analysis (Part I)

Mihir Rao

Contents

1 Introduction: Integer Multiplication and Merge Sort **2**

1.1 Integer multiplication and the cost of a step . . . 2

1.2 Recursive multiplication and Karatsuba's idea . . . 2

1.3 Divide and conquer through sorting . . . 3

1.4 Merging two sorted arrays . . . 4

1.5 Counting work in one merge . . . 5

1.6 The recursion-tree analysis . . . 5

1.7 Three principles for algorithm analysis . . . 7

2 Asymptotic Analysis **9**

2.1 Counting loops before simplifying . . . 9

2.2 Big-O: an eventual upper bound . . . 10

2.3 Polynomial examples and fixed witnesses . . . 11

2.4 Omega and Theta: lower and matching bounds . . . 11

2.5 Little-o and the order of the quantifiers . . . 13

2.6 Constants inside an exponent . . . 13

2.7 A sum and a maximum have the same growth . . . 14

Lecture 1: Introduction: Integer Multiplication and Merge Sort

An algorithm specifies how to turn an input into a required output. Its design asks which steps to use; its analysis asks whether those steps are correct and how much work they require. Better algorithms can make a task feasible even when faster hardware alone would not. Algorithmic thinking also gives a way to study processes such as markets and evolution.

Two familiar tasks, multiplying integers and sorting numbers, introduce a recurring question: can we improve on the obvious method?

1.1 Integer multiplication and the cost of a step

The input is two nonnegative integers x and y, each represented with n decimal digits; the required output is xy. Here a primitive operation adds or multiplies two single digits. Multiplying two arbitrarily long integers is therefore not one primitive operation.

The grade-school method forms one partial product for each digit of y, then adds the shifted rows. For the given example,

$$\begin{array}{r} 5678 \\ \times 1234 \\ \hline 22712 \\ 170340 \\ 1135600 \\ 5678000 \\ \hline 7006652 \end{array}$$

There are n rows, each involving work proportional to n, including digit multiplication and carrying. The final additions also take at most a constant times n^2 work. Thus the total grows quadratically with the number of digits. Doubling the digit count roughly quadruples this work.

1.2 Recursive multiplication and Karatsuba's idea

Split each input into a high half and a low half. With $m = n/2$ for an even digit count, write

$$x = 10^m a + b, \qquad y = 10^m c + d,$$

where each block uses at most m digits, allowing leading zeros. Expanding the product gives

$$xy = 10^{2m} ac + 10^m (ad + bc) + bd. \tag{1.1}$$

This identity reduces the original task to products of smaller integers. A direct recursive method computes ac, ad, bc, bd separately, then combines them using equation (1.1). Small inputs are handled directly.

Karatsuba multiplication reuses information to remove one of these four recursive products. Compute

$$u = ac, \qquad v = bd, \qquad w = (a + b)(c + d).$$

Since $w = ac + ad + bc + bd$, the middle coefficient is $w - u - v = ad + bc$. Therefore

$$\boxed{xy = 10^{2m}u + 10^{m}(w - u - v) + v.}$$

There are three recursive multiplications, plus additions, subtractions, and decimal shifts. Sums such as $a + b$ can have an extra carry digit; a complete implementation must preserve it. The smaller number of recursive products is the reason to investigate this method further, rather than counting a recursive call as a single inexpensive step.

Example: For $x = 5678$ and $y = 1234$, take $a = 56$, $b = 78$, $c = 12$, $d = 34$. Then

$$u = 56 \cdot 12 = 672, \qquad v = 78 \cdot 34 = 2652,$$
$$w = 134 \cdot 46 = 6164, \quad w - u - v = 2840.$$

Recombining gives

$$10^4(672) + 10^2(2840) + 2652 = 7006652,$$

which agrees with the grade-school calculation.

1.3 Divide and conquer through sorting

A divide and conquer algorithm splits a task into smaller subproblems, solves them recursively, and combines their answers. The combination step matters: reducing the input size is useful only if the work of solving and combining the subproblems is controlled.

Definition 1.1 [Sorting problem]. Given an array of n distinct numbers, return the same numbers in increasing order. Correctness requires both ordering and preservation of the input elements.

Merge sort splits the array into two halves, sorts each half recursively, and merges the sorted halves. Arrays of length zero or one are already sorted.

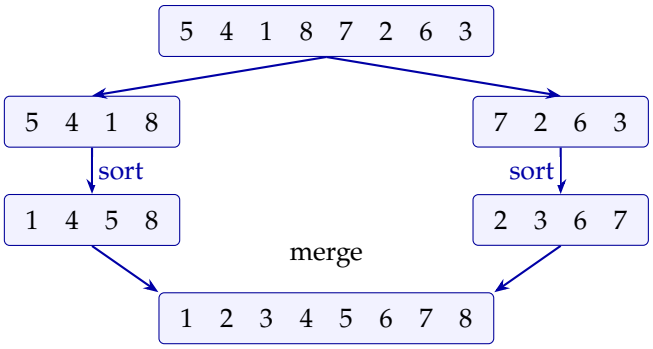


Figure 1: Merge sort on the supplied eight-element array. The two recursive calls return sorted halves; the final merge combines them in increasing order.

Algorithm 1.1: Merge sort

Input: Array X of length n, with distinct entries.
Output: The entries of X in increasing order.

```
if n ≤ 1 then
    return X
m ← ⌊n/2⌋
A ← MergeSort(X[1 : m])
B ← MergeSort(X[m + 1 : n])
return Merge(A, B)
```

Both recursive arrays have length smaller than n when $n \geq 2$, so the procedure eventually reaches a base case. The floor handles odd lengths without discarding an element.

1.4 Merging two sorted arrays

The smallest unused element of a sorted array is its first unused element. To find the next output element, it is therefore enough to compare the two current front elements. After choosing one, advance only its pointer. If an array is exhausted, copy the remaining elements of the other array.

Algorithm 1.2: Merge two sorted arrays

Input: Increasing arrays $A[1 : p]$ and $B[1 : q]$, with distinct entries overall.
Output: An increasing array C containing every input entry once.

```
i ← 1; j ← 1; C ← an empty array
while i ≤ p and j ≤ q do
    if A[i] < B[j] then
        Append A[i] to C; i ← i + 1
    else
        Append B[j] to C; j ← j + 1
Append the remaining suffix A[i : p], if nonempty, to C
Append the remaining suffix B[j : q], if nonempty, to C
return C
```

Lemma 1.2 [Correctness of merging]. *If A and B are sorted, the merge procedure returns their entries in increasing order, with no loss or repetition.*

Proof. The idea is to maintain that the output contains exactly the smallest entries processed so far. Initially it is empty. While both arrays have unused entries, the smaller front element is the smallest unused element of their union, because neither sorted suffix contains a smaller element behind its front. Appending that element preserves the ordering and moves exactly one pointer. When one array is exhausted, the remaining sorted suffix contains all remaining elements, each at least as large as the last element appended. Copying it finishes the output. Every element is copied once, so the result has the required order and contents. □

Proposition 1.3 [Correctness of merge sort]. *Merge sort returns the input entries in increasing order.*

Proof. Use induction on the array length. Lengths zero and one are base cases. For $n \geq 2$, each half is shorter than the input, so the induction hypothesis guarantees that the recursive calls sort both halves correctly. The merge lemma then orders their union and preserves all entries. This proves the claim for length n. □

1.5 Counting work in one merge

Let $m = p + q$ be the total number of input entries. Each iteration copies one entry, and the suffix copies account for exactly the remaining entries. Thus merging takes work proportional to m.

For a coarse operation count, use the simplified loop accounting: two pointer initializations and at most four elementary steps for each output position. This gives

$$\text{MergeCost}(m) \leq 4m + 2 \leq 6m \qquad (m \geq 1).$$

The constant depends on which bookkeeping steps are charged. Completing the end cases in actual code can change it, but does not change the linear growth. This bound is a mathematical accounting model, not a count of machine instructions.

1.6 The recursion-tree analysis

Each merge is linear in its own input size, but there are many merges. The useful observation is that the total input size across any one level of recursion is n.

Assume first that $n = 2^h$, so all splits are even. A recursion tree has one node per recursive call. At depth j there are 2^j subproblems, each of size $n/2^j$. The leaves have size one, so they occur at depth $h = \log_2 n$. There are $h + 1$ levels, counting the root and the leaves.

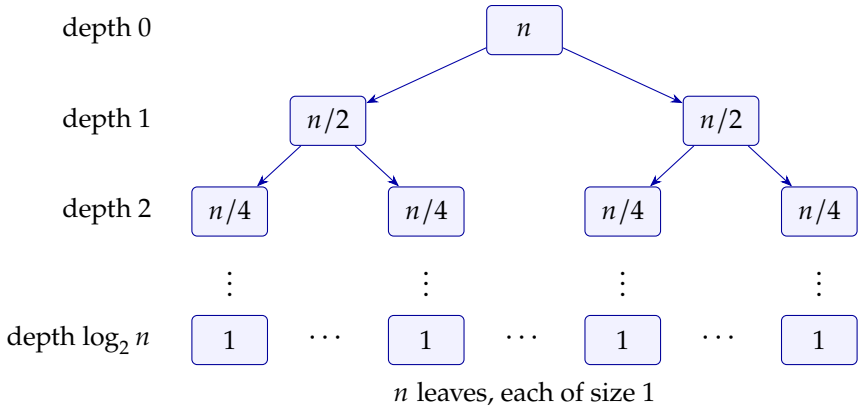


Figure 2: Schematic recursion tree for n a power of two. The number of subproblems doubles as their sizes halve. Intermediate levels are omitted.

Theorem 1.4 [Merge-sort running time]. *In the coarse operation model above, merge sort on $n = 2^h \geq 1$ entries uses at most $6n \log_2 n + 6n$ charged operations, including a constant charge for each leaf.*

Proof. Bound the work one level at a time. Charge at most $6m$ for a node of size m, with leaf work bounded by 6. The charge at depth j is at most

$$\underbrace{2^j}_{\text{number of nodes}} \quad \underbrace{6\left(\frac{n}{2^j}\right)}_{\text{work per node}} = 6n.$$

There are $\log_2 n + 1$ levels. Summing their charges gives

$$\sum_{j=0}^{\log_2 n} 6n = 6n(\log_2 n + 1) = 6n \log_2 n + 6n.$$

The leaf charge is included, even though a leaf performs no merge. □

For a general positive n, balanced splitting has at most $\lceil \log_2 n \rceil + 1$ levels. Nodes at any fixed depth cover disjoint portions of the input, whose total size is at most n. The same reasoning gives a bound proportional to $n \log n$. The power-of-two assumption makes the displayed level counts exact; it is not required by the algorithm.

1.7 Three principles for algorithm analysis

1. Worst-case analysis gives a running-time bound for every input of a given size. It is useful for a general-purpose routine whose future inputs are unknown. An average-case guarantee needs a model for which inputs are likely; a benchmark measures selected instances.
2. Suppress constant factors and lower-order terms when comparing growth rates. Constants depend on implementation and computing hardware; retaining all of them can obscure the main design difference.
3. Asymptotic analysis studies how the cost grows as the input becomes large. A smaller growth rate need not give a smaller cost on every small input.

For example, compare the illustrative costs

$$f(n) = 6n \log_2 n + 6n, \qquad g(n) = \tfrac{1}{2}n^2.$$

The linearithmic expression eventually becomes much smaller, although the quadratic expression is smaller over part of the range of small n. These are cost formulas, not measured runtimes.

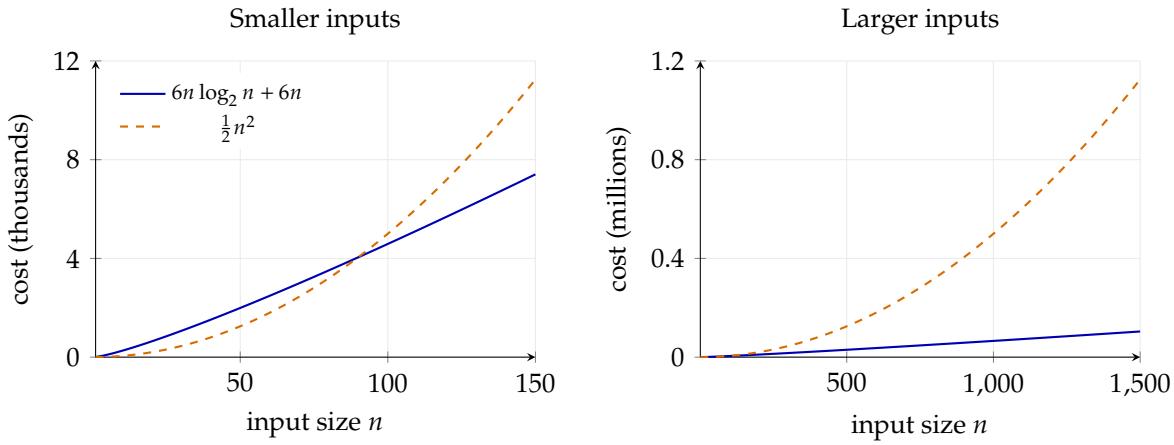


Figure 3: The supplied illustrative cost functions at two input scales. The relative advantage of $6n \log_2 n + 6n$ grows with the input size. The vertical axes use different units.

The aim is a worst-case cost that grows slowly with input size, often as close to linear as the task permits. Merge sort illustrates why this viewpoint helps: its modest per-level work is repeated only logarithmically many times.

Revision. Specify the input, output, and primitive operations before counting work. Karatsuba obtains the cross term from three recursive products. Merge sort uses a linear merge at each node; the total work at each recursion depth is linear, and there are logarithmically many depths. Correctness follows by combining a merge invariant with induction.
Common mistakes: charging one step for a large-integer product; forgetting a carry digit in $a + b$; reading past an exhausted merge input; counting $\log_2 n$ levels while also including the leaves; or treating an asymptotic comparison as a timing claim for every input size.
Check yourself: How does $(a + b)(c + d)$ eliminate one multiplication? Why is the smaller front element safe to output? Why is the work per tree level independent of its depth? Which assumption makes the level counts exact, and which parts of the algorithm work without it?

Source. Tim Roughgarden, Algorithms: Design and Analysis (Part I), Introduction.

Lecture 2: Asymptotic Analysis

How can we compare algorithms without fixing a processor, programming language, and compiler? Count elementary operations as a function of the input size, then focus on how that count grows. Asymptotic analysis suppresses constant factors and lower-order terms while retaining the differences that become significant on large inputs.

For example, a bound such as $6n \log_2 n + 6$ has the growth rate $n \log n$. The notation $O(n \log n)$ makes this comparison precise. Here n denotes the input size, such as the length of an array, and $T(n)$ usually denotes the worst-case running time over inputs of that size.

2.1 Counting loops before simplifying

Count each array comparison and loop update as constant work. First identify how many times that work is performed. A return statement can end an algorithm early, but a worst-case bound must also cover inputs on which no early return occurs.

Algorithm 2.1: Search one array

Input: Array $A[1 : n]$ and target t.
Output: Whether A contains t.

```
1 for i ← 1 to n do
2 |   if A[i] = t then
3 |   |   return true
4 return false
```

If t is absent, all n entries are compared with it. There is constant work per iteration, so the worst-case running time grows linearly with n. Finding t at the first position is a faster input, but does not reduce the worst-case bound.

Algorithm 2.2: Search two arrays in succession

Input: Arrays $A[1 : n]$, $B[1 : n]$, and target t.
Output: Whether either array contains t.

```
1 for i ← 1 to n do
2 |   if A[i] = t then
3 |   |   return true
4 for i ← 1 to n do
5 |   if B[i] = t then
6 |   |   return true
7 return false
```

When t is absent from both arrays, there are $n + n = 2n$ comparisons. The loops run in succession, so their work adds. Suppressing the factor 2 leaves the same linear growth rate as the first search.

Algorithm 2.3: Check two arrays for a common entry

Input: Arrays $A[1:n]$ and $B[1:n]$.
Output: Whether the arrays share an entry.

```
for i ← 1 to n do
    for j ← 1 to n do
        if A[i] = B[j] then
            return true
return false
```

If the arrays have no common entry, each of the n outer iterations executes all n inner iterations. There are n^2 comparisons, giving quadratic worst-case growth.

Algorithm 2.4: Check an array for duplicates

Input: Array $A[1:n]$.
Output: Whether two different positions have the same entry.

```
for i ← 1 to n do
    for j ← i + 1 to n do
        if A[i] = A[j] then
            return true
return false
```

Starting the inner loop at $i+1$ avoids comparing an entry with itself or checking a pair twice. If all entries are distinct, the comparison count is

$$\sum_{i=1}^{n}(n-i) = (n-1) + (n-2) + \cdots + 1 = \frac{n(n-1)}{2}.$$

The leading term is $n^2/2$, so the growth is still quadratic. A changing inner-loop length requires summing the iteration counts; it does not by itself imply a faster asymptotic bound.

2.2 Big-O: an eventual upper bound

Let $T(n) \geq 0$ be a cost function, and let $f(n) > 0$ for sufficiently large positive integers n. We want to express that T eventually stays below a fixed multiple of f, even if the two functions behave differently on small inputs.

Definition 2.1 [Big-O]. Write $T(n) = O(f(n))$ if there exist constants $c > 0$ and $n_0 \geq 1$ such that

$$0 \leq T(n) \leq cf(n) \qquad \text{for every integer } n \geq n_0.$$

This is an asymptotic upper bound.

The fixed multiplier c absorbs constant factors, and n_0 excludes small inputs. Neither depends on n; the bound holds for every integer $n \geq n_0$.

An upper bound need not be tight: a linear running time is also $O(n^2)$. When comparing the loop examples, $O(n)$ and $O(n^2)$ describe their respective linear and quadratic growth, rather than the only upper bounds they satisfy.

2.3 Polynomial examples and fixed witnesses

Proposition 2.2 [Polynomial upper bound]. *For fixed real coefficients $a_0, \dots, a_k$ and an integer $k \geq 0$, let $p(n) = a_k n^k + \cdots + a_1 n + a_0$. Then $|p(n)| = O(n^k)$.*

Proof. The idea is to bound every lower power by the highest power. For $n \geq 1$ and $0 \leq i \leq k$, we have $n^i \leq n^k$. Hence

$$|p(n)| \leq \sum_{i=0}^{k} |a_i| n^i \leq \left(\sum_{i=0}^{k} |a_i| \right) n^k.$$

Choose $n_0 = 1$ and $c = \max\{1, \sum_{i=0}^{k} |a_i|\}$. These are fixed witnesses for the big-O definition. In particular, the bound applies to a nonnegative polynomial running time. □

Proposition 2.3 [Different polynomial degrees]. *For every integer $k \geq 1$, n^k is not $O(n^{k-1})$.*

Proof. The idea is that one uncancelled factor of n eventually exceeds every fixed constant. If $n^k = O(n^{k-1})$, there would be constants $c, n_0 > 0$ with $n^k \leq c n^{k-1}$ for all integers $n \geq n_0$. Dividing by the positive quantity n^{k-1} gives $n \leq c$ for all such n. Choose an integer $n > \max\{n_0, c\}$. This contradicts the required inequality. □

The first proof establishes a bound by exhibiting witnesses. The second rules out a bound by showing that every proposed pair of witnesses fails on a sufficiently large input.

2.4 Omega and Theta: lower and matching bounds

Big-O bounds a function from above. A corresponding lower bound says that the cost eventually remains at least a fixed multiple of another function.

Definition 2.4 [Big-Omega]. Write $T(n) = \Omega(f(n))$ if there exist constants $c > 0$ and $n_0 \geq 1$ such that

$$T(n) \geq c f(n) \qquad \text{for every integer } n \geq n_0.$$

This is an asymptotic lower bound.

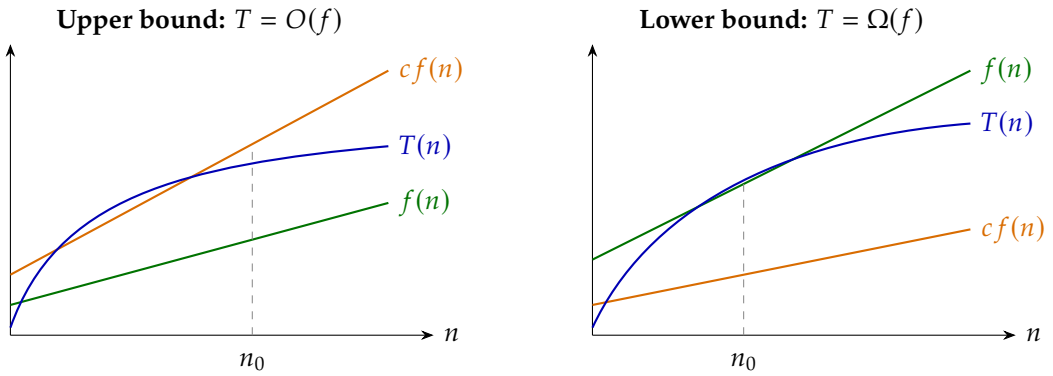


Figure 4: Schematic eventual bounds. For big-O, T lies below cf after n_0; for big-Omega, it lies above cf. The unscaled function f need not lie on the same side of T as its scaled version.

Definition 2.5 [Big-Theta]. Write $T(n) = \Theta(f(n))$ if both $T(n) = O(f(n))$ and $T(n) = \Omega(f(n))$. Equivalently, there exist constants $c_1, c_2 > 0$ and $n_0 \geq 1$ such that

$$c_1 f(n) \leq T(n) \leq c_2 f(n) \qquad \text{for every integer } n \geq n_0.$$

This is an asymptotically tight bound.

To obtain one threshold from separate upper and lower bounds, take the larger of their two thresholds. Then both inequalities hold simultaneously. Conversely, the two-sided inequality immediately supplies both bounds.

These symbols describe inequalities between functions. They do not select which inputs are considered: if $T(n)$ is a worst-case running time, then $\Omega(f(n))$ is a lower bound on that same worst-case function.

Example: Consider $T(n) = \frac{1}{2}n^2 + 3n$. For every integer $n \geq 1$,

$$\tfrac{1}{2}n^2 \leq T(n) \leq \tfrac{1}{2}n^2 + 3n^2 \leq 4n^2.$$

Thus $T(n) = \Theta(n^2)$, with $c_1 = 1/2$, $c_2 = 4$, and $n_0 = 1$. The other comparisons follow directly:

Claim	True?	Reason for $n \geq 1$
$T(n) = O(n)$	No	$T(n)/n = \frac{1}{2}n + 3$ is unbounded.
$T(n) = \Omega(n)$	Yes	$T(n) \geq \frac{1}{2}n^2 \geq \frac{1}{2}n$.
$T(n) = \Theta(n^2)$	Yes	$\frac{1}{2}n^2 \leq T(n) \leq 4n^2$.
$T(n) = O(n^3)$	Yes	$T(n) \leq 4n^2 \leq 4n^3$.

The cubic upper bound is valid, but the quadratic Theta bound describes the growth more precisely. Similarly, the four loop examples have tight worst-case bounds $\Theta(n)$, $\Theta(n)$, $\Theta(n^2)$, and $\Theta(n^2)$, respectively.

2.5 Little-o and the order of the quantifiers

A fixed upper multiplier allows two functions to grow at the same rate. To say that T eventually becomes smaller than *every* positive multiple of f, use a stronger comparison.

Definition 2.6 [Little-o]. Write $T(n) = o(f(n))$ if for every constant $c > 0$, there exists a threshold $n_0 \geq 1$ such that

$$0 \leq T(n) \leq cf(n) \qquad \text{for every integer } n \geq n_0.$$

For big-O, one suitable multiplier is enough. For little-o, the inequality must eventually work no matter how small the positive multiplier is. Here n_0 may depend on the chosen c, but not on the running variable n.

Proposition 2.7. *For every integer $k \geq 1$, $n^{k-1} = o(n^k)$.*

Proof. The idea is to make the ratio $1/n$ smaller than any given constant. Fix $c > 0$, and choose $n_0 = \max\{1, \lceil 1/c \rceil\}$. Then every integer $n \geq n_0$ satisfies $1/n \leq c$, so

$$n^{k-1} \leq cn^k.$$

Since this works for every positive c, the little-o definition holds. □

2.6 Constants inside an exponent

A constant added to an exponent behaves differently from a constant multiplied by the input size in that exponent.

Proposition 2.8. *$2^{n+10} = O(2^n)$, but 2^{10n} is not $O(2^n)$.*

Proof. Compare each expression with 2^n by division. First,

$$2^{n+10} = 2^{10}2^n = 1024 \cdot 2^n,$$

so $c = 1024$ and $n_0 = 1$ give the required upper bound.
If $2^{10n} = O(2^n)$, there would be fixed constants $c, n_0 > 0$ with $2^{10n} \leq c2^n$ for all integers $n \geq n_0$. Dividing by 2^n yields $2^{9n} \leq c$. Choose an integer $n > \max\{n_0, (\log_2 c)/9\}$. Then $2^{9n} > c$, a contradiction. □

The factor 10 in $10n$ cannot be discarded: after division, it leaves a ratio that grows without bound. The fixed additive shift +10 leaves only a constant ratio.

2.7 A sum and a maximum have the same growth

The larger of two nonnegative quantities controls their sum up to a factor of two. The identity is useful even when the two functions cross repeatedly, so neither is always the larger one.

Proposition 2.9 [Maximum and sum]. *For positive functions f and g on the positive integers,*

$$\max\{f(n), g(n)\} = \Theta(f(n) + g(n)).$$

Proof. Use pointwise inequalities, so no assumption about which function grows faster is needed. Each function is at most the maximum, and both are nonnegative. Therefore, for every $n \geq 1$,

$$\max\{f(n), g(n)\} \leq f(n) + g(n) \leq 2\max\{f(n), g(n)\}.$$

Rearranging gives

$$\tfrac{1}{2}\big(f(n) + g(n)\big) \leq \max\{f(n), g(n)\} \leq f(n) + g(n).$$

These are the Theta inequalities with $c_1 = 1/2$, $c_2 = 1$, and $n_0 = 1$. □

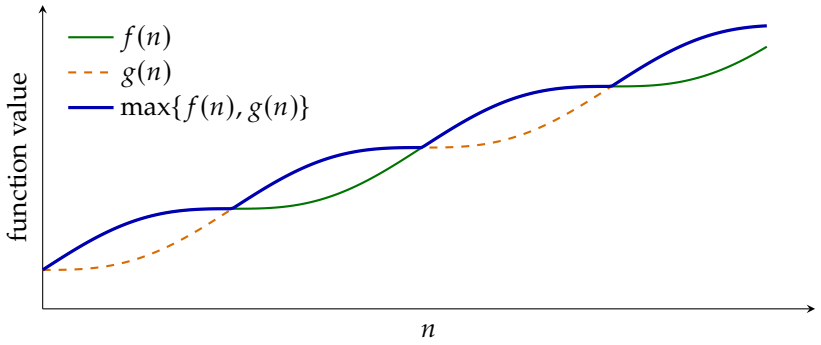


Figure 5: Schematic crossing functions. Their pointwise maximum follows whichever curve is higher. Their sum remains between this maximum and twice the maximum, including where the curves switch order.

Revision. Count the work before suppressing constants: successive loops add, while nested loops require counting all inner iterations. O gives an eventual upper bound, Ω a lower bound, and Θ matching upper and lower bounds. Little-o requires an eventual upper bound for every positive multiplier.
Common mistakes: allowing witnesses to depend on n; treating every big-O bound as tight; confusing a bound's direction with the choice of worst-case or best-case inputs; or discarding a multiplier inside an exponent.
Check yourself: Why are two successive searches linear? Which constants witness the polynomial upper bound? How do the quantifiers in big-O and little-o differ? Why can two functions cross infinitely often without invalidating the maximum/sum comparison?

Source. Tim Roughgarden, Algorithms: Design and Analysis (Part I), Asymptotic Analysis.